Lecture 17: Complexity Classes, P, NP, and Polynomial-Time Reductions

Reference: Chapters 1–2, Arora & Barak

Complexity Classes

Complexity classes are classes of languages defined by the resources (time, space, ...) required to compute them.

Definition 1.1 (DTIME($T(n)$))**.** For a function $T : \mathbb{N} \to \mathbb{N}$, the complexity class DTIME($T(n)$) is the set of languages (functions) for which there exists a constant $c > 0$ and a Turing Machine M so that M runs in time $c \cdot T(n)$ on inputs of length n and computes L.

That is, it is the set of functions computable in time $O(T(n))$.

"D" stands for "deterministic."

The Class P

Definition 2.1 (P)**.**

$$P = \bigcup_{k \geq 1} \mathrm{DTIME}(n^k)$$

P is the class of all languages/functions computable in **polynomial time**.

In this unit, P will represent problems with **efficient algorithms**.

Objections:

- n^{100} isn't efficient!
- For some use cases, n^3 isn't efficient.
- Worst-case analysis isn't useful...

Why P anyway?

- *Empirically:* if we find a poly-time algorithm for a problem, we often find one with a reasonable exponent.
- *Mathematically:* P is closed under many natural operations. A polynomial of a polynomial is a polynomial. It is closed under any subroutines.
- *"Robust":* Does not depend on the details of the definition of TM.

The Class EXP

Definition 3.1 (EXP)**.**

$$\mathrm{EXP} = \bigcup_{c \geq 1} \mathrm{DTIME}(2^{n^c})$$

EXP is the class of functions computable in **exponential (of a polynomial in n) time**.

Relation between P and EXP:

$$P \subseteq \text{EXP}$$

It is **known** that $P \neq \text{EXP}$, i.e., $\text{EXP} \not\subseteq P$. This gives a *separation* between P and EXP.

The Class NP

Definition 4.1 (NP — via certificates)**.** The class **NP** is the set of languages L such that there exists a polynomial $p(n)$ and a polynomial-time Turing Machine M (the *verifier*) so that:

$$x \in L \iff \exists\, u \in \{0,1\}^{p(|x|)} \text{ such that } M(x,u) = 1.$$

In other words:

- If $x \in L$, then $\exists$ a string u with $M(x,u) = 1$.
- If $x \notin L$, then $M(x,u) = 0$ for all $u \in \{0,1\}^{p(|x|)}$.

We call u the **certificate** (or witness), and M the **verifier**.

Examples of NP Problems

Example 4.1 (INDSET)**.** INDSET(G,k): Does G have an independent set of size k?

INDSET $\in$ NP. *Proof:* The certificate is the list of k vertices in an independent set. The verifier checks there are no edges between vertices in this set.

Example 4.2 (TSP)**.** **TSP**: Is there a tour of length $\leq T$?

Certificate: the order of points in the tour. Verifier: add all distances and check the total is $\leq T$.

Example 4.3 (COMPOSITE)**.** COMPOSITE(N): Given integer N, is N a composite number?

Certificate: integers $x, y > 1$ such that $xy = N$. (Note: PRIME(N) is *not* so simple to see as NP.)

Example 4.4 (Graph Isomorphism)**.** Given graphs G, H: is there a permutation π of the vertices of H such that $G = H_\pi$?

In NP: certificate is the permutation π; check entries of adjacency matrices.

Example 4.5 (Factoring)**.** Given integers N, L, U: does N have a factor in the interval $[L, U]$?

In NP: certificate is a factor x; check if x divides N and $L \leq x \leq U$.

Remark. We note that examples (1) and (2) are believed to be "harder" than (4) and (5) — but what does this mean precisely? We will formalize this with reductions.

$P \subseteq NP$

Theorem 4.1. *If $L \in P$, then $L \in NP$.*

Proof. Set $p(n) = 0$, $u = \emptyset$; M just computes L directly. So $P \subseteq NP$. □

It was shown that PRIME $\in P$ (via a complicated algorithm) and is known to be in NP.

The "N" in NP

"N" stands for **non-deterministic**.
Definition 4.2 (NTIME($T(n)$))**.** The class NTIME($T(n)$) is the class of languages for which there exists a constant c and a TM M (running in time $c \cdot T(n)$) with *two* transition functions δ_1, δ_2 so that $x \in L$ if and only if there exists a sequence of choices of δ_1, δ_2 so that on input x, M reaches state q_{accept} in time $c \cdot T(n)$.

If $x \notin L$, then M does not reach q_{accept} on any sequence of choices of δ_1, δ_2.

Equivalence: Certificate $u \iff$ sequence of choices.
Definition 4.3 (NP — via non-determinism)**.**

$$NP = \bigcup_{k \geq 1} \text{NTIME}(n^k).$$

We have seen:

$$P \subseteq NP \subseteq \bigcup_{k \geq 1} \text{DTIME}(2^{n^k}) = EXP.$$

Major Open Question. Does $P = NP$?

Polynomial-Time Reductions

How do we relate problems to each other with respect to polynomial-time computation? With **reductions**.
Definition 5.1 (Karp Reduction, $A \leq_p B$)**.** We say a language $A \subseteq \{0,1\}^*$ is **polynomial-time (Karp) reducible** to $B \subseteq \{0,1\}^*$ if there is a polynomial-time computable function $g : \{0,1\}^* \to \{0,1\}^*$ so that

$$x \in A \iff g(x) \in B.$$

We write $A \leq_p B$.

If f_A, f_B are the boolean functions associated to A, B, then:

$$f_B(g(x)) = f_A(x).$$

Fact 5.1. *If $A \leq_p B$ and $B \in P$, then $A \in P$.*

Intuitively, $A \leq_p B$ means "B is at least as hard as A."

NP-Hardness and NP-Completeness

Definition 6.1 (NP-hard)**.** We say a language L is **NP-hard** if $A \leq_p L$ for every $A \in NP$.
Definition 6.2 (NP-complete)**.** We say a language L is **NP-complete** if L is NP-hard *and* $L \in NP$.
Theorem 6.1 (Properties of Reductions)**.**

1. ***Transitivity:*** *If $A \leq_p B$ and $B \leq_p C$, then $A \leq_p C$.*
   Why? Apply the composition of functions and use that a polynomial of a polynomial is a polynomial.
2. *If A is NP-hard and $A \in P$, then $P = NP$.*
3. *If A is NP-complete, then $P = NP$ iff $A \in P$.*
   Implication: If there are NP-complete problems, we can solve P vs. NP just by solving one problem.

Existence of NP-Complete Problems

Theorem 7.1. *There exist NP-complete problems.*

Proof sketch. (Full proof in the textbook.) The language is

$$\text{TMSAT}\big(\langle\alpha\rangle,\, x,\, 1^n,\, 1^t\big) :$$

Does there exist $u \in \{0,1\}^n$ such that $M_\alpha(x,u) = 1$ and $M_\alpha(x,u)$ runs in at most t steps?

(Here $\langle\alpha\rangle$ is the code of a TM and x is the input.)

This may seem unnatural, but it fits the TM definition of NP directly. □

SAT and 3-SAT

Theorem 8.1 (Cook–Levin)**.** *The problems **SAT** and **3-SAT** are NP-complete.*

Boolean Satisfiability

Boolean variables $x_1, x_2, \ldots, x_n$, each taking values True or False.

- $\neg x_i$ ("not x_i") is a *literal*, as is x_i itself.
- A **CNF clause** is an OR of k literals, e.g.:

  $$(x_1 \vee x_3 \vee \neg x_7)$$

  which evaluates to True if at least one literal is true.
- A **CNF formula** (Conjunctive Normal Form) is the AND of CNF clauses, e.g.:

  $$(x_1 \vee \neg x_2) \;\wedge\; (x_5 \vee x_3) \;\wedge\; (x_2 \vee \neg x_6 \vee x_{11}).$$

Definition 8.1 (SAT)**.** Given a boolean formula in CNF, does there exist an assignment of True/False to the n variables so that the formula evaluates to True?
Definition 8.2 (3-SAT)**.** SAT where each clause has exactly 3 literals.

SAT and 3-SAT are very natural and **expressive**: we will see how to reduce many NP problems to SAT or 3-SAT.